Code

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None


# Insert a node
def insert(root, key):
    if root is None:
        return Node(key)

    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)

    return root


# Search for a key
def search(root, key):
    if root is None:
        return False

    if root.key == key:
        return True
    elif key < root.key:
        return search(root.left, key)
    else:
        return search(root.right, key)


# Find the minimum value node
def find_min(root):
    current = root

    while current.left is not None:
        current = current.left

    return current


# Delete a node
def delete(root, key):
    if root is None:
        return root
```

```
    if key < root.key:
        root.left = delete(root.left, key)

    elif key > root.key:
        root.right = delete(root.right, key)

    else:
        # Node with no child
        if root.left is None and root.right is None:
            return None

        # Node with only right child
        if root.left is None:
            return root.right

        # Node with only left child
        if root.right is None:
            return root.left

        # Node with two children
        temp = find_min(root.right)
        root.key = temp.key
        root.right = delete(root.right, temp.key)

    return root


# Display BST using inorder traversal
def display(root):
    if root is not None:
        display(root.left)
        print(root.key, end=" ")
        display(root.right)


# Main program
root = None

while True:
    print("\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
```

```
    key = int(input("Enter key: "))
    root = insert(root, key)
    print("Key inserted.")

elif choice == 2:
    key = int(input("Enter key to search: "))

    if search(root, key):
        print("Key found.")
    else:
        print("Key not found.")

elif choice == 3:
    key = int(input("Enter key to delete: "))

    if search(root, key):
        root = delete(root, key)
        print("Key deleted.")
    else:
        print("Key not found.")

elif choice == 4:
    print("BST (Inorder): ", end="")
    display(root)
    print()

elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice.")
```

Output

```
--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key: 50
Key inserted.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key: 30
Key inserted.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key: 70
Key inserted.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key: 20
Key inserted.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
```

```
Enter key: 40
Key inserted.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4
BST (Inorder): 20 30 40 50 70

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 2
Enter key to search: 40
Key found.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 2
Enter key to search: 90
Key not found.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 3
Enter key to delete: 30
Key deleted.

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4
```

```
BST (Inorder): 20 40 50 70

--- BINARY SEARCH TREE ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 5
Program terminated.
```